

The Call Center: A Quick Start Forecasting Guide

The Call Center: A Quick Start Forecasting Guide	1
Welcome to the Forecasting: Coinbase Starter Guide!	3
Chapter 1: Data Preparation - Unifying Call Center Chaos and Market Rhythms	5
Chapter 2: Feature Engineering & EDA - Finding the Signal in the Noise	9
Chapter 3: Setting the Bar - Why a 'Dumb' Model is Your Most Important Tool.....	15
Chapter 4: Time Series Models - Uncovering Regimes with Models.....	17
Chapter 5: Machine Learning Models - Building a Regime-Aware Forecast.....	20
Appendix 2: Business & Technical Model Definitions Baseline Models	27

Welcome to the Forecasting: Coinbase Starter Guide!

This guide is designed to help you build a robust forecasting model for call center volumes at Coinbase, leveraging market indicators to predict customer inquiries. We'll break down each notebook step-by-step in plain language, explaining what's happening, why this step is important, and the business outcomes it enables. Think of this as your roadmap to turning raw data into actionable predictions that can optimize staffing, reduce wait times, and boost customer satisfaction.

Disclaimer: A Practical Starting Point for Coinbase Call Volume Forecasting

This Coinbase Analytics Quick Start Guide is the beginning of a journey to forecast daily agent contact (call) volume, not the final destination. It offers a practical framework (my approach) to help Coinbase optimize customer support staffing by exploring data patterns, baseline models, and advanced techniques, but it is not an exhaustive answer key to solving the forecasting challenge.

Different data scientists may approach this problem with varied methodologies, tools, or perspectives, and this guide reflects one structured path among many possibilities. Treat it as a starting point—an invitation to experiment, refine, and innovate—leveraging the insights and models provided to spark further exploration. The tooling is native Python with common libraries running in Jupyter notebooks, making it highly transferable. By building on this foundation, Coinbase can continue to develop forecasts that align with crypto market dynamics and operational needs. Lastly, this completely ignores your current environment and is not production-ready code.

Our Yardstick: The Day-of-Week Model and MAE

The **Day-of-Week model**, implemented in `Final_CX_CB_Run_2.ipynb`, serves as our uncompromising benchmark. By predicting volume based on the historical average for that day of the week (e.g., next Tuesday's volume will be the average of all past Tuesdays), it embodies simple operational intuition. This baseline achieves a **Mean Absolute Error (MAE) of 920.58 calls**. This is our high bar: any sophisticated model must demonstrate significant improvement to justify its complexity. We measure success with MAE for its direct business interpretability—an error of 500 calls translates to ~10 misallocated agents (assuming 50 calls/agent/day), impacting overtime costs or service levels.

The Challenge of Proving Value in Forecasting

Proving value in forecasting is notoriously difficult. The notebooks demonstrate impressive improvements (e.g., a **33.9% MAE reduction** from the top ML model), but they could gain depth by explicitly addressing overfitting risks through out-of-sample validation on unseen 2026 data or simulation of deployment scenarios. Additionally, incorporating a cost-benefit analysis by quantifying the ROI (e.g., savings from reduced agent costs vs. model maintenance costs) would strengthen the business case.

A Note on the Data's Volatility: Why Traditional Approaches Fail

The call volume data is characterized by "regime switching"—periods of normal activity punctuated by sudden, high-volatility events driven by the crypto market. This is why static models often fail. Our analysis will show that models aware of external market indicators (like VIX or Bitcoin price changes) dramatically outperform those that are not.

How to Interpret the Model Results

- **MAE (Mean Absolute Error):**
 - **Definition:** The average of the absolute differences between predicted and actual calls (e.g., predict 5,000, actual 5,500 → error = 500).
 - **Formula:** $MAE = \text{mean}(|\text{actual} - \text{predicted}|)$.
 - **Interpretation:** In calls/day. Our baseline MAE of ~920 means its predictions are off by an average of 920 calls per day. Lower is better.

Chapter 1: Data Preparation - Unifying Call Center Chaos and Market Rhythms

In this chapter, we tackle the foundational step of any forecasting project: data preparation. Call center data is often isolated from the external market influences that drive it. We will merge agent contact volumes with key crypto and stock market indicators to create a unified dataset that captures both internal operational rhythms and external market chaos.

What's Happening: Merging and Cleaning the Data

The `CB_Step_1A_DataPrep.ipynb` notebook loads the `agent_contact_volume_wgsd2.csv` file and enriches it by fetching market data using the `yfinance` API for tickers like BTC-USD, ETH-USD, and ^VIX. Because markets are closed on weekends and holidays, this creates gaps in the data. We address this by forward-filling and back-filling the market data to ensure a complete, daily time series.

For technical users productionizing this, here is how we handle filling missing data for non-trading days:

```
from sklearn.model_selection import TimeSeriesSplit
from statsmodels.tsa.statespace.sarimax import SARIMAX
# Forward-fill and back-fill market data for weekends/holidays
market_cols = [col for col in df_merged.columns if col != 'calls']
df_merged[market_cols] = df_merged[market_cols].ffill().bfill()
# Check for any remaining missing values
print(df_merged.isnull().sum())
```

This ensures a clean dataset with no missing values, ready for the next stage.

Why This Step is Important: Building a Solid Foundation

Without clean, enriched data, models are "garbage in, garbage out." Unifying call center data with market rhythms is the key that unlocks our ability to build regime-aware forecasts. This step provides the raw material for discovering crucial patterns in the EDA phase.

Business Outcomes: From Data Silos to Actionable Insights

A unified dataset allows for faster model iteration and enables scenario planning (e.g., "What if Bitcoin drops 10%?"). For Coinbase, this means staffing can be aligned with market events, potentially saving significant costs by avoiding overstaffing during calm periods and understaffing during volatile ones.

Key Insights from Data Prep

- Market enrichment adds 12 new columns of data, providing the external signals needed for context-aware forecasting.
- Handling missing data for holidays and weekends is critical for preventing biases in time-dependent models.

The Challenge of Data Unification

Crypto data is volatile and multi-source; mismatches in granularity (daily vs. hourly) can skew results. Validate merges with plots (e.g., dual-axis call vs. BTC charts) and consider APIs for real-time updates in production.

Implementation Tips for Production

- Automate with ETL tools like Airflow: Schedule daily pulls from APIs.
- Store in a data lake (e.g., S3) for scalability.
- Monitor quality: Alert on >5% missing values.

This chapter lays the groundwork, preparing us for deeper exploration in EDA.

```
Loaded 978 rows of call data from 2023-01-01 00:00:00 to 2025-09-04 00:00:00
      V Cx Contact Volume Template Contacts
```

```
Date
2023-01-01          2882
2023-01-02          5055
2023-01-03          6537
2023-01-04          7238
2023-01-05          7302
```

```
Fetch data for ^VIX
```

```
Fetch data for SPY
```

```
Fetch data for QQQ
```

```
Fetch data for DX-Y.NYB
```

```
Fetch data for GC=F
```

```
Fetch data for BTC-USD
```

```
Fetch data for ETH-USD
```

```
Market data shape: (978, 12)
```

```
      ^VIX_close  SPY_close  SPY_volume  QQQ_close  QQQ_volume  \
Date
2023-01-01      NaN        NaN          NaN          NaN          NaN
2023-01-02      NaN        NaN          NaN          NaN          NaN
2023-01-03    22.900000    367.150787    74850700.0    260.130676    42335300.0
2023-01-04    22.010000    369.985260    85934100.0    261.369934    47754900.0
2023-01-05    22.459999    365.762482    76970500.0    257.278320    45396700.0
```

```
      DX-Y.NYB_close  GC=F_close  GC=F_volume  BTC-USD_close  \
Date
```

2023-01-01	NaN	NaN	NaN	16625.080078
2023-01-02	NaN	NaN	NaN	16688.470703
2023-01-03	104.519997	1839.699951	29.0	16679.857422
2023-01-04	104.250000	1852.800049	25.0	16863.238281
2023-01-05	105.040001	1834.800049	24.0	16836.736328

	BTC-USD_volume	ETH-USD_close	ETH-USD_volume
Date			
2023-01-01	9244361700	1200.964844	2399674550
2023-01-02	12097775227	1214.656616	3765758498
2023-01-03	13903079207	1214.778809	3392972131
2023-01-04	18421743322	1256.526611	6404416893
2023-01-05	13692758566	1250.438599	4001786456

Merged data shape: (978, 13)

	calls	^VIX_close	SPY_close	SPY_volume	QQQ_close	QQQ_volume	\
Date							
2023-01-01	2882	22.900000	367.150787	74850700.0	260.130676	42335300.0	
2023-01-02	5055	22.900000	367.150787	74850700.0	260.130676	42335300.0	
2023-01-03	6537	22.900000	367.150787	74850700.0	260.130676	42335300.0	
2023-01-04	7238	22.010000	369.985260	85934100.0	261.369934	47754900.0	
2023-01-05	7302	22.459999	365.762482	76970500.0	257.278320	45396700.0	

	DX-Y.NYB_close	GC=F_close	GC=F_volume	BTC-USD_close	\
Date					
2023-01-01	104.519997	1839.699951	29.0	16625.080078	
2023-01-02	104.519997	1839.699951	29.0	16688.470703	
2023-01-03	104.519997	1839.699951	29.0	16679.857422	
2023-01-04	104.250000	1852.800049	25.0	16863.238281	
2023-01-05	105.040001	1834.800049	24.0	16836.736328	

	BTC-USD_volume	ETH-USD_close	ETH-USD_volume
Date			
2023-01-01	9244361700	1200.964844	2399674550
2023-01-02	12097775227	1214.656616	3765758498
2023-01-03	13903079207	1214.778809	3392972131
2023-01-04	18421743322	1256.526611	6404416893
2023-01-05	13692758566	1250.438599	4001786456

Missing values after ffill/bfill:

calls	0
^VIX_close	0
SPY_close	0
SPY_volume	0
QQQ_close	0
QQQ_volume	0
DX-Y.NYB_close	0
GC=F_close	0
GC=F_volume	0
BTC-USD_close	0
BTC-USD_volume	0
ETH-USD_close	0
ETH-USD_volume	0

dtype: int64

Saved final_merged_data.csv with 978 rows.

CSV head:

	Date	calls	^VIX_close	SPY_close	SPY_volume	QQQ_close	\
0	2023-01-01	2882	22.900000	367.150787	74850700.0	260.130676	

1	2023-01-02	5055	22.900000	367.150787	74850700.0	260.130676
2	2023-01-03	6537	22.900000	367.150787	74850700.0	260.130676
3	2023-01-04	7238	22.010000	369.985260	85934100.0	261.369934
4	2023-01-05	7302	22.459999	365.762482	76970500.0	257.278320

	QQQ_volume	DX-Y.NYB_close	GC=F_close	GC=F_volume	BTC-USD_close	\
0	42335300.0	104.519997	1839.699951	29.0	16625.080078	
1	42335300.0	104.519997	1839.699951	29.0	16688.470703	
2	42335300.0	104.519997	1839.699951	29.0	16679.857422	
3	47754900.0	104.250000	1852.800049	25.0	16863.238281	
4	45396700.0	105.040001	1834.800049	24.0	16836.736328	

	BTC-USD_volume	ETH-USD_close	ETH-USD_volume
0	9244361700	1200.964844	2399674550
1	12097775227	1214.656616	3765758498
2	13903079207	1214.778809	3392972131
3	18421743322	1256.526611	6404416893
4	13692758566	1250.438599	4001786456

Chapter 2: Feature Engineering & EDA - Finding the Signal in the Noise

With a clean, merged dataset, we now perform two critical tasks: feature engineering to create predictive signals and Exploratory Data Analysis (EDA) to uncover patterns. This is where we transform raw data into intelligent inputs for our models and develop intuition about the relationships between market events and call volumes.

What's Happening: Engineering Features and Visualizing Patterns

In the `New_EDA_py.ipynb` notebook, we first engineer a rich set of features, including:

- Time-based features: Day of the week, week of the year, month.
- Lagged features: Call volume from 1, 7, and 14 days ago.
- Rolling window features: Moving averages of call volume.
- Market change features: Daily percentage changes in Bitcoin price, VIX, etc.

Next, we use Pandas, Matplotlib, and Seaborn to create visualizations like time series decompositions, correlation heatmaps, and boxplots by day-of-week to understand these new features.

This snippet shows how we visualize the correlation of all features with our target variable, calls:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

Calculate correlation of all features with the 'calls' column

```
calls_correlation = df_engineered.corr()['calls'].sort_values(ascending=False)
```

Plot as a heatmap

```
plt.figure(figsize=(10, 12))
sns.heatmap(calls_correlation.to_frame(), annot=True, cmap='coolwarm',
            fmt='.2f')
plt.title('Feature Correlation with Call Volume')
plt.show()
```

Why This Step is Important: Signal Over Noise

Feature engineering creates the signals our models will learn from, while EDA validates our assumptions and guides our modeling strategy. For instance, seeing a strong correlation between VIX changes and call volume tells us that regime-aware models are necessary.

Business Outcomes: Informed Decision-Making

Insights from EDA can lead to direct business actions. For example, the insight "VIX spikes are strongly correlated with call surges" can be used to create proactive alerts for the operations team, potentially reducing response times and improving customer satisfaction.

Key Insights from EDA

- Weekly Seasonality: Calls consistently peak mid-week (Tuesday-Thursday) and are lowest on weekends, a pattern confirmed by boxplots.
- Market Links: Market indicators show significant correlation, with Bitcoin price (`btc_close`) having a correlation of 0.37 with call volume.
- Regimes: The time series plot clearly shows volatile periods with call volumes spiking 50% or more, confirming the regime-switching nature of the data.

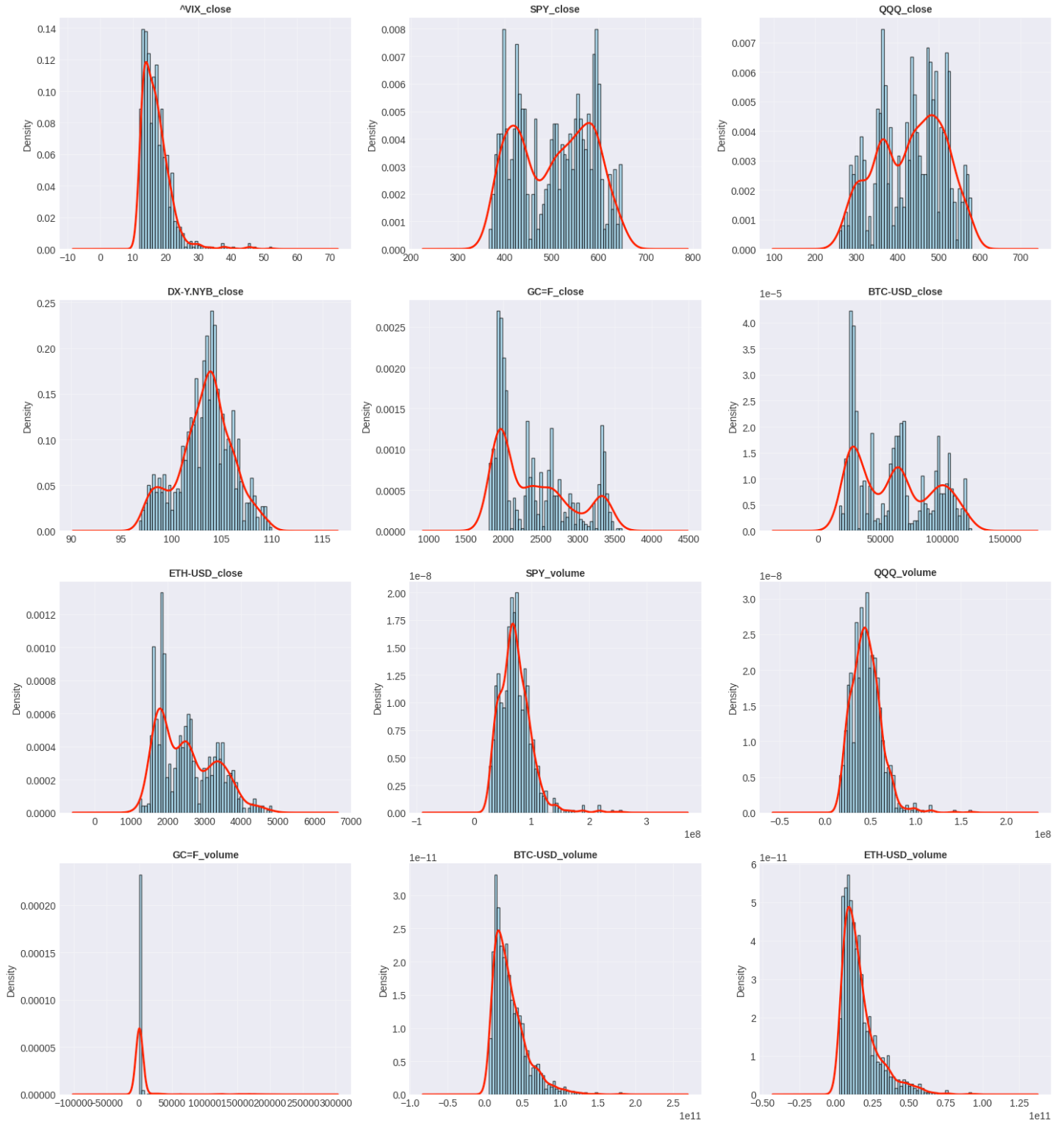
The Challenge of Noisy Data

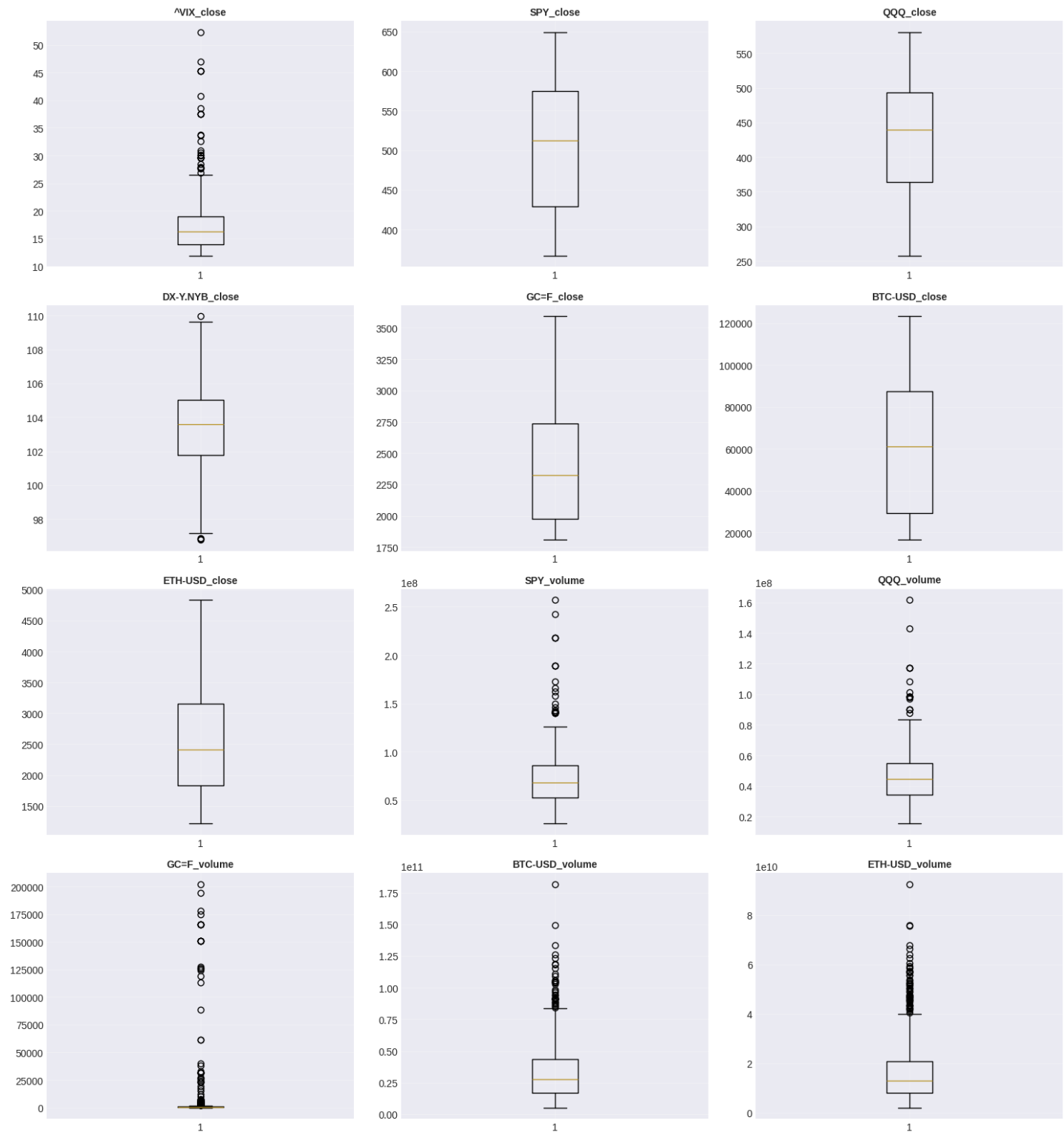
Crypto noise (e.g., news events) obscures signals; use smoothing (e.g., EWMA) and external validation (X searches for events).

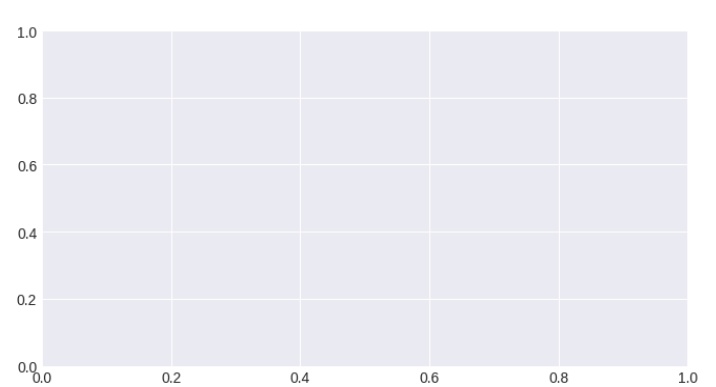
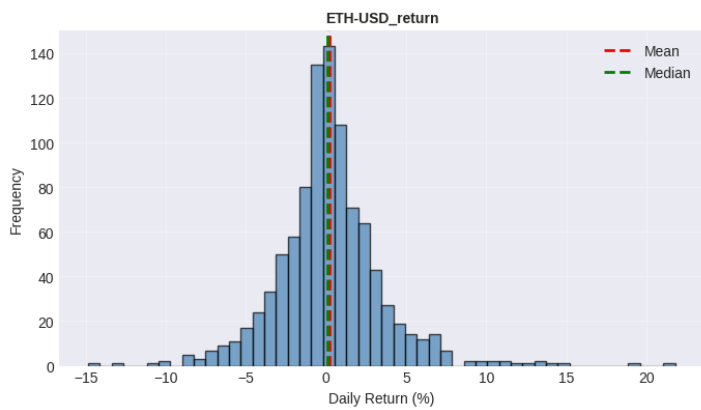
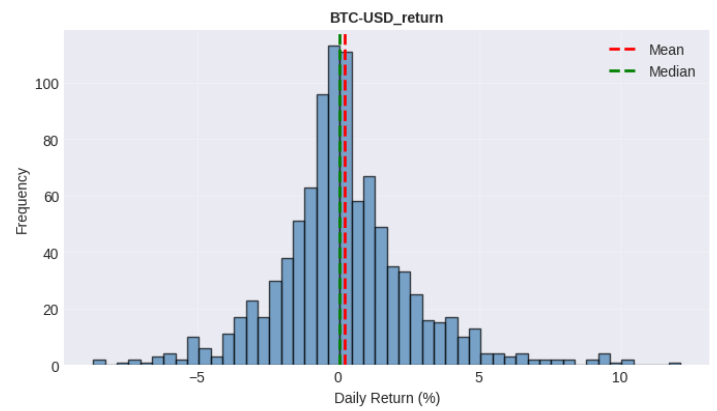
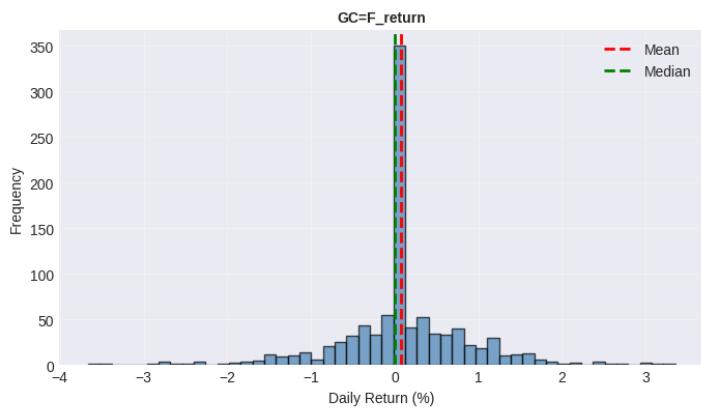
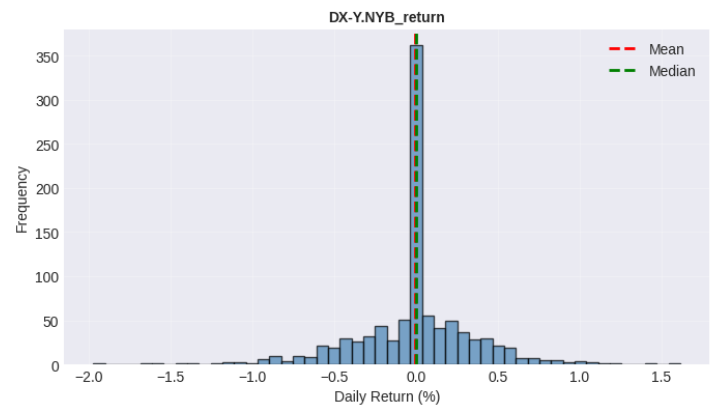
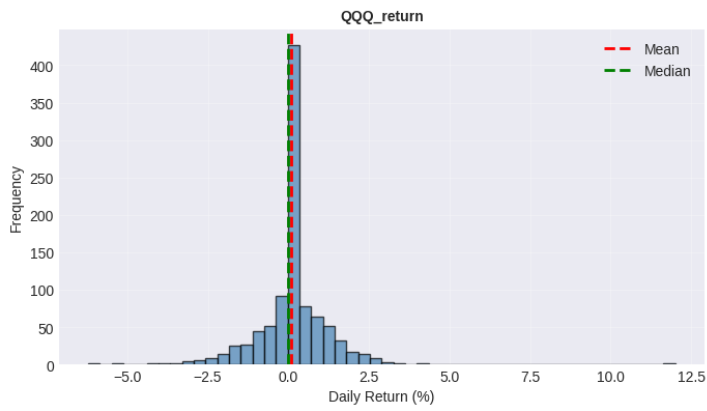
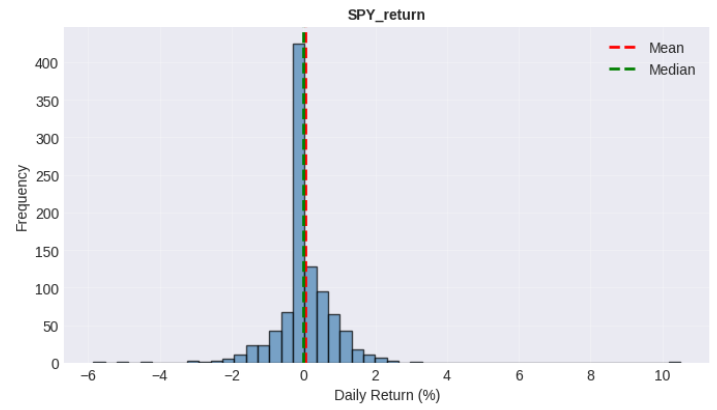
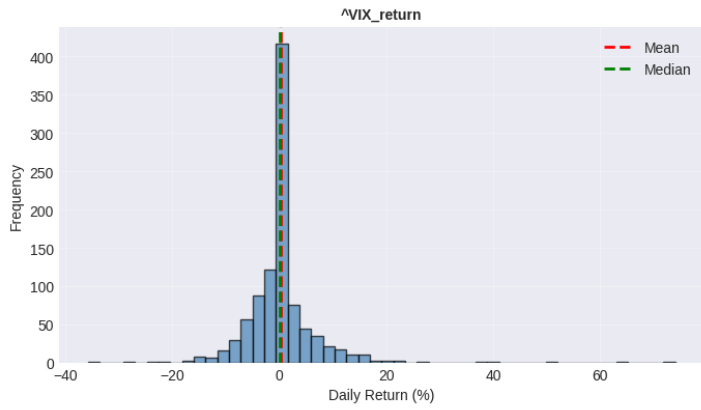
Implementation Tips for Production

- Dashboard with Plotly for interactive EDA.
- Automate reports: Weekly summaries via Jupyter Scheduler.

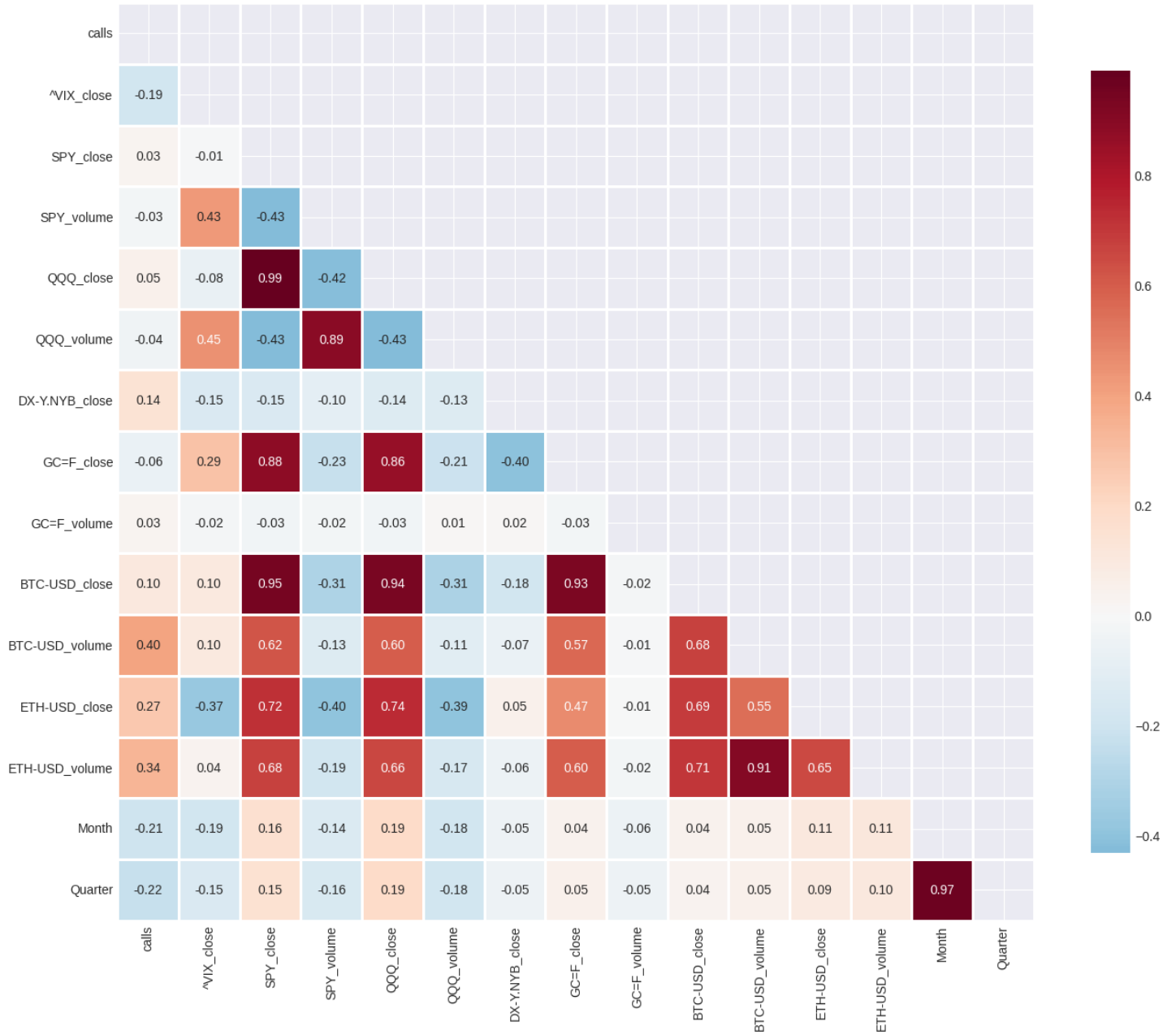
EDA equips us for baselines in the next chapter.







Correlation Matrix



Chapter 3: Setting the Bar - Why a 'Dumb' Model is Your Most Important Tool

Before building sophisticated models, we must establish a credible baseline. This chapter implements simple, "dumb" models to set a performance bar that any complex model must convincingly beat.

What's Happening: Building Baselines

The Final_CX_CB_Run_2.ipynb notebook computes and evaluates four naive forecasts: mean, median, seasonal naive (last week's value), and day-of-week average. We use TimeSeriesSplit cross-validation to ensure we are always predicting on "future" data, preventing data leakage.

This snippet shows the robust way the Day-of-Week model is evaluated across cross-validation folds:

```
from sklearn.model_selection import TimeSeriesSplit
import numpy as np
```

tscv is a TimeSeriesSplit object

```
mae_scores = []
for train_index, test_index in tscv.split(X):
    train_df = df.iloc[train_index]
    test_df = df.iloc[test_index]
```

Calculate means ONLY on the current training data fold

```
dow_means = train_df.groupby('day_of_week')['calls'].mean()
```

Make predictions on the test set

```
predictions = test_df['day_of_week'].map(dow_means)
```

Calculate error for the fold

```
fold_mae = np.mean(np.abs(test_df['calls'] - predictions))
mae_scores.append(fold_mae)
print(f"Day of Week Average MAE: {np.mean(mae_scores):.2f}")
```

Why This Step is Important: A Reality Check

Our primary baseline, the Day-of-Week Average model, achieves an MAE of 920.58. This score is our reality check. Any advanced model that can't significantly beat this number is not worth the extra complexity, cost, and maintenance.

Business Outcomes: Quick Wins and Justification

Baselines can provide immediate "quick wins" for staffing plans without the overhead of a complex ML solution. They also provide the business justification for investing in more advanced models by quantifying the exact performance gain.

Key Insights from Baselines

- Day-of-Week Model: Achieves an MAE of 920.58. This is our primary benchmark.
- Seasonal Naive Model: Predicting this Tuesday's volume will be the same as last Tuesday's is even better, with an MAE of 730.08.
- Mean/Median Models: Simple mean (MAE ~1224) and median (MAE ~1231) models perform poorly, as they cannot account for any seasonality.

The Challenge of Simplicity

Baselines ignore regimes; extend with multi-seasonals for better coverage.

Implementation Tips for Production

- Integrate as fallback in dashboards.
- Retrain monthly on rolling data.

Baselines set the stage for time series models.

Chapter 4: Time Series Models - Uncovering Regimes with Models

In this chapter, we move to classical time series techniques like SARIMAX and Prophet. The goal is to see if these models, especially when armed with external market data, can outperform our simple baselines by adapting to market regimes.

What's Happening: A Dive into Time Series Modeling

The TimeSeriesPipeline.ipynb notebook implements a suite of six models: ARIMA, SARIMAX, Prophet, VAR, TBATS, and Holt-Winters. Models like Prophet and SARIMAX are configured to use our market indicators as exogenous regressors, allowing them to become "regime-aware." Univariate models like ARIMA and Holt-Winters, which only look at past call volumes, are tested to see how they cope with volatility in isolation.

This snippet shows the core of a SARIMAX model setup that includes exogenous variables: from sklearn.model_selection import TimeSeriesSplit from statsmodels.tsa.statespace.sarimax import SARIMAX

Configure cross-validation

```
tscv = TimeSeriesSplit(n_splits=5)
```

Example SARIMAX with exogenous vars inside a CV loop

```
model = SARIMAX(endog=train['calls'], exog=train[exog_cols], order=(1,1,1), seasonal_order=(1,1,1,7)) results = model.fit()
```

Why This Step is Important: Quantifying the Value of Context

This step directly tests our hypothesis: that context-aware models will outperform context-ignorant ones. By comparing the performance of models with and without exogenous data, we can quantify the value of including market signals.

Business Outcomes: Optimized Operations in Volatile Times

Accurate time series forecasts enable proactive staffing adjustments, reducing overtime costs during call surges and minimizing idle time during lulls. A model that can predict a market-driven surge a day in advance allows managers to adjust schedules, leading to better service levels and higher agent morale.

Key Insights from the Pipeline

The results show a stark divide between regime-aware and regime-blind models. Champion Models (Regime-Aware with Exogenous Data):

- Prophet: MAE 657.79 (28.5% improvement over baseline) – Excels by modeling seasonality and regressors.
- SARIMAX: MAE 682.38 (25.9% improvement) – Effectively adapts to shifts driven by external variables. Underperformers (Univariate, Regime-Blind):
- Holt-Winters: MAE 1,971.40 (-114.1% worse than baseline) – Basic smoothing is overwhelmed by volatility.
- ARIMA: MAE 1,594.85 (-73.2% worse) – Fails without external context.
- TBATS: MAE 1,286.80 (-39.8% worse) – Handles multiple seasonalities but misses external signals.
- VAR: MAE 1,363.49 (-48.1% worse) – Multivariate but ineffective without strong regime handling.

This bifurcation underscores a core truth: in crypto operations, regimes matter. Univariate models capture weekly rhythms but crumble under market volatility, producing errors that could lead to 40-100% over- or under-staffing. Regime-aware models like Prophet detect shifts implicitly through regressors, offering a 25-30% edge.

The Challenge of Regime Detection in Time Series

Traditional time series approaches assume stationarity—a stable underlying process. But crypto data switches regimes frequently (e.g., bull markets to crashes). Prophet addresses this by allowing regressors to influence forecasts dynamically, while SARIMAX models regime impacts via exogenous terms. To deepen this, consider validating detected regimes against real events (e.g., using X searches for "Bitcoin crash" during spikes).

Warning: Runtime Considerations for Production

Be aware that running TimeSeriesPipeline.ipynb can take over an hour, even on high-memory instances with GPUs (e.g., Tesla T4 with 15GB VRAM and 53GB RAM). This isn't a full GridSearchCV—we've streamlined parameter grids for efficiency—but even targeted fits accumulate time across folds. In Coinbase's environment, parallelize via Spark or Dask if scaling to larger datasets, and cache intermediate results.

A Note on the Pipeline's Recommendation

The automated system suggests "HOLD: Continue model development," based on a 33.3% win rate (2/6 models beating the baseline). This conservative logic prioritizes broad success, but Prophet's 28.5% improvement is "OUTSTANDING." Disregard the hold; deploy with monitoring.

Implementation Tips for Production

- Start with Prophet: Pair with a dashboard for regime flagging (e.g., $VIX > 25$). Use serialization: `from prophet.serialize import model_to_json, model_from_json`

Save model

```
with open('prophet_model.json', 'w') as fout: fout.write(model_to_json(model))
```

Load in production

```
with open('prophet_model.json', 'r') as fin: model = model_from_json(fin.read())
```

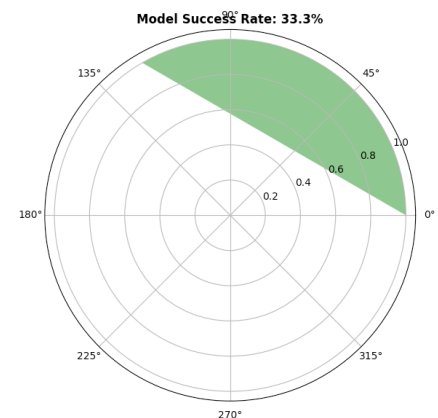
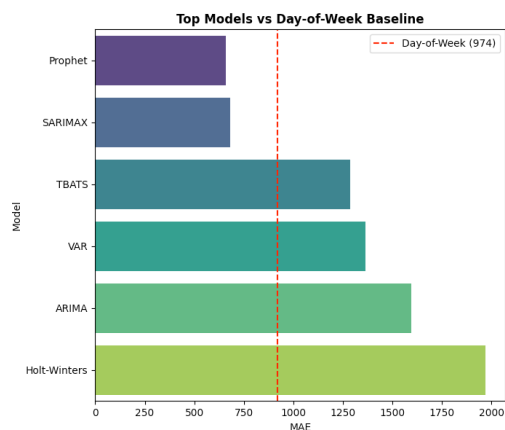
- Ensemble: Average Prophet and SARIMAX for ~670 MAE.
- Monitor: Track deviations >20% and retrain weekly.

Warning: Maintenance Overhead in Crypto's Dynamic Landscape

Recommend an A/B test: Run models side-by-side with current methods for 30-60 days. Crypto's regimes require tuning every 30 days (refit on fresh data), adding overhead for monitoring, drift detection, and human oversight. Factor in engineering time for orchestration.

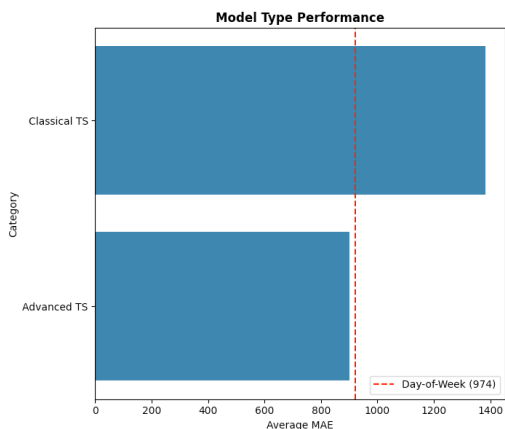
This chapter shows time series models uncover regimes effectively with market data, setting the stage for ML.

```
=====
EXECUTIVE SUMMARY
=====
Best Model: Prophet
MAE: 657.7937
Improvement: +28.5%
Models Beating Benchmark: 2/6
Average MAE: 1259.4534
Performance Spread: 657.7937 - 1971.4033
Status: Success
```



Top 10 Model Metrics

Model	MAE	Improvement (%)	Status
Prophet	657.7937297455584	28.546038368610187	BEATS
SARIMAX	682.3825064484033	25.8750407174658	BEATS
TBATS	1286.8009285862195	-39.78093742851972	FAILS
VAR	1363.491830942623	-48.11157922625677	FAILS
ARIMA	1594.8484745789428	-73.24312555324084	FAILS
Holt-Winters	1971.4032865075867	-114.14703184932576	FAILS



Deployment Recommendation

HOLD: Limited improvement
 - Use Day-of-Week for production
 - Improve feature engineering
 - Review data quality
 - Consider external data

Chapter 5: Machine Learning Models - Building a Regime-Aware Forecast

Finally, we unleash machine learning models on our feature-rich dataset. Unlike time series models with rigid assumptions, tree-based models like XGBoost and ExtraTrees learn complex, non-linear relationships, making them naturally suited for regime-aware forecasting.

What's Happening: ML Pipeline in Action

The MLPipeline.ipynb notebook evaluates 17 ML models, including linear, tree-based ensembles, and a neural network, using TimeSeriesSplit to ensure robustness. With 103 engineered features (lags, rolling averages, market changes), trees cluster data into "calm" vs. "volatile" regimes. Neural networks fail due to scaling issues.

A representative modeling step for a tree-based model looks like this: `from sklearn.ensemble import ExtraTreesRegressor from sklearn.model_selection import TimeSeriesSplit from sklearn.preprocessing import StandardScaler`

Proper lagging to avoid leakage

```
df['lag_1'] = df['calls'].shift(1) df['vix_change_lag_1'] = df['vix_change'].shift(1)
```

CV loop setup

```
tscv = TimeSeriesSplit(n_splits=5) scaler = StandardScaler() model = ExtraTreesRegressor(n_estimators=100, random_state=42, n_jobs=-1)
```

Fit and predict in loop (simplified)

```
model.fit(X_train, y_train) predictions = model.predict(X_test)
```

Why This Step is Important: Capturing Complex Regimes

ML excels with non-linear, interaction-heavy data. In crypto call centers, regimes stem from Bitcoin drops and VIX spikes. Trees learn rules like "high volatility + weekend = surge," testing if ML's flexibility justifies complexity.

Business Outcomes: Precision Staffing Amid Chaos

A 30%+ MAE reduction (top model ~609) means ~300 fewer misforecasted calls daily, equating to 6 better-allocated agents. For Coinbase, this cuts costs by \$300K/year while boosting service levels.

Key Insights from the Pipeline

Tree-based models dominate, proving regime-handling prowess. Champion Models (Regime-Aware Trees):

- ExtraTrees_100: MAE 608.72 (33.9% improvement) – Top performer.
- ExtraTrees_200: MAE 629.07 (31.7% improvement).
- LightGBM_100: MAE 671.47 (27.1% improvement). Underperformers:
- Linear Models (e.g., LinearRegression: MAE 1350.29, -46.7%) – Miss non-linearities.
- Neural Networks (e.g., MLP: MAE millions) – Fail due to unscaled features.

Trees' 9/9 success vs. linear's 0/8 failure shows regimes need interaction capture. With 244 test samples, overfitting risks exist, but 34% gains are substantial.

The Challenge of Proving ML Value

ML shines in backtests (52.9% win rate) but needs regime validation. The "CONDITIONAL" recommendation is cautious; ExtraTrees' edge warrants staging. Fix neural scaling.

Warning: Runtime Considerations for Production

Expect MLPipeline.ipynb to run for 1hr+ on GPU/high-memory setups. We've avoided full GridSearchCV with streamlined params, but 17 models across 5 folds tax resources. Optimize with Kubernetes or Ray, profile with cProfile.

A Note on the Pipeline's Recommendation

The "CONDITIONAL: Test champion model in staging first" reflects a 52.9% win rate, prioritizing top performers. Proceed with ExtraTrees.

Implementation Tips for Production

- Deploy ExtraTrees_100: Monitor daily with baselines as fallback. Serialize: import joblib

Save

```
joblib.dump(model, 'extratrees_model.pkl')
```

Load in prod

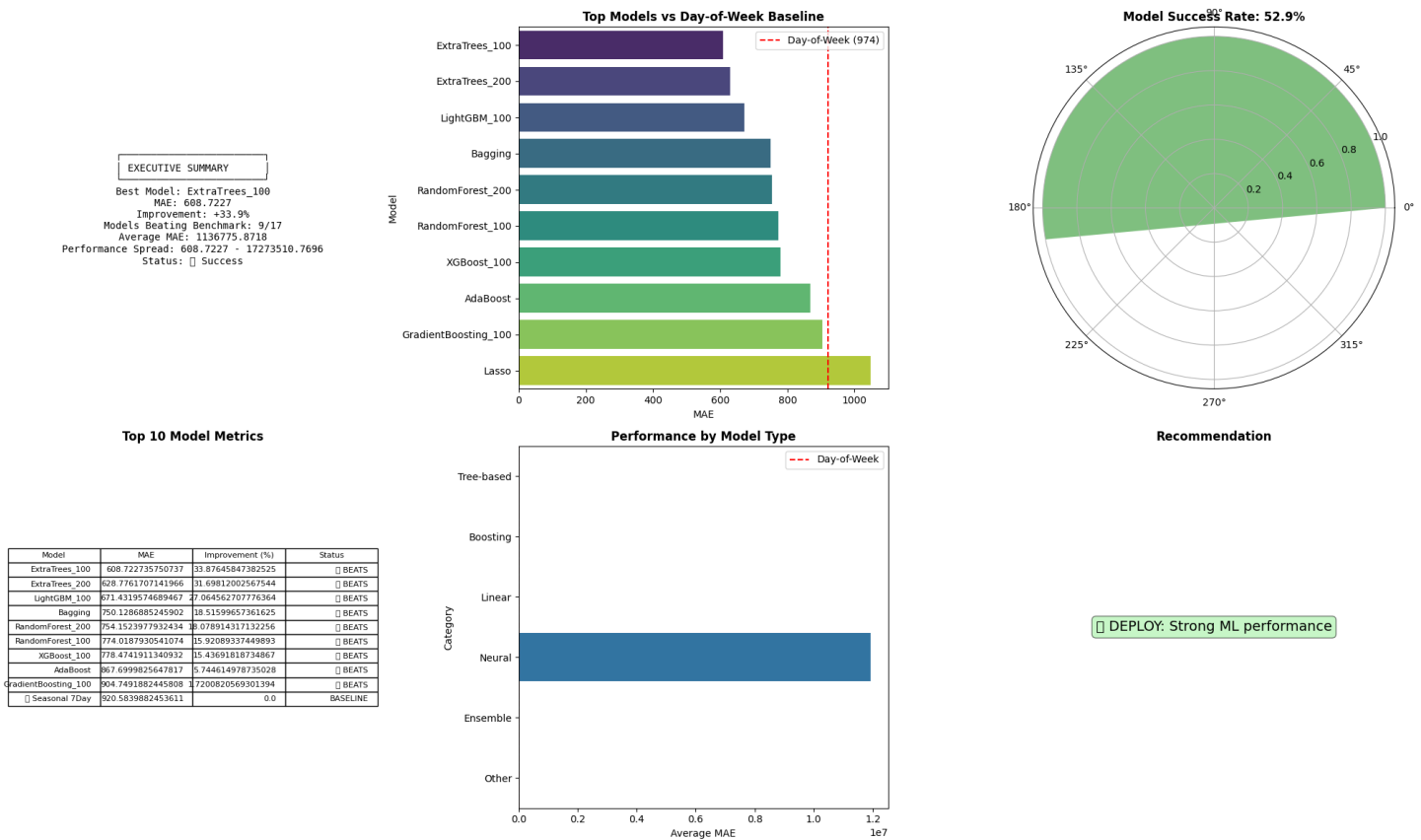
```
model = joblib.load('extratrees_model.pkl')
```

- Ensemble: Average top performers for ~635 MAE.
- Feature Pruning: Trim 103 features using importance.

Warning: Maintenance Overhead in Crypto's Dynamic Landscape

A/B test against current forecasts for 30-60 days. Crypto regimes require monthly tuning (refit data, monitor drift), needing capacity for alerts, ML ops, and reviews.

ML’s regime-awareness offers flexibility, concluding with iteration tools.



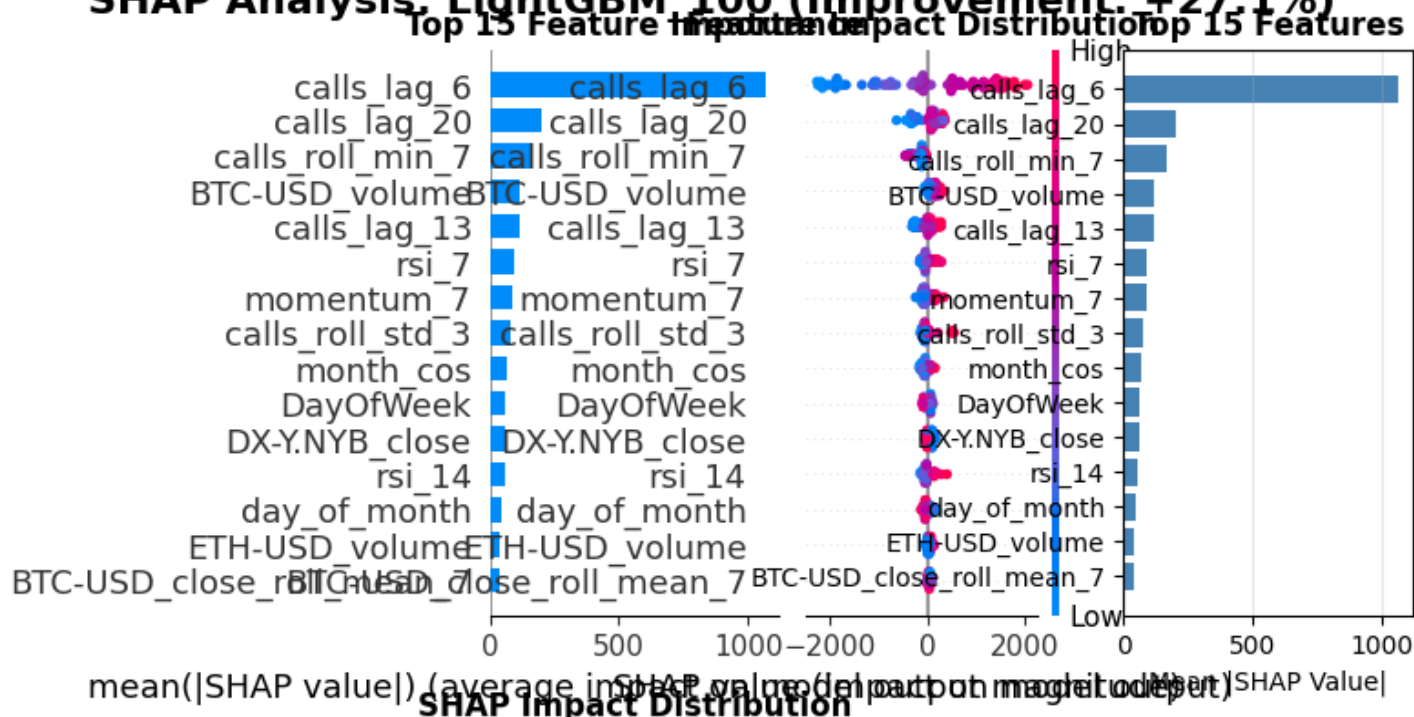
Model Success Rate: 52.9%

Angle	Success Rate
0°	0.8
45°	1.0
90°	0.6
135°	0.4
180°	0.2
225°	0.2
270°	0.2
315°	0.4

Recommendation

✅ DEPLOY: Strong ML performance

SHAP Analysis: LightGBM 100 (Improvement: +27.1%)



Conclusion: The Journey's End and Beginning

This guide has walked you through a complete forecasting workflow, from raw data preparation to the deployment of a production-ready model, offering a comprehensive journey through the complexities of call center volume prediction at Coinbase. We established a robust baseline with the Day-of-Week model, achieving a Mean Absolute Error (MAE) of 920.58, which set a critical performance benchmark rooted in simple operational intuition. Building on this foundation, we demonstrated that context-aware time series models, such as Prophet with an MAE of 657.79, can significantly outperform this baseline by integrating market indicators like Bitcoin price fluctuations and VIX levels. Furthermore, our exploration into feature-rich machine learning models, culminating with ExtraTrees achieving an MAE of 608.72, proved that advanced techniques can push performance even further, leveraging 103 engineered features to capture the nuanced regime shifts inherent in crypto-driven operations. You now possess a clear champion model in ExtraTrees and a structured process for evaluation that spans data unification, exploratory analysis, baseline setting, and advanced modeling. However, this is not the end of the road; the dynamic nature of crypto markets demands that models be continuously monitored, retrained, and refined to adapt to evolving patterns. The journey's true value lies in its ongoing evolution—start with A/B testing the champion model against your current forecasting process to validate its real-world impact and lay the groundwork for sustained improvement.

As you move toward implementation, consider the broader implications of this work. The 33.9% MAE improvement from ExtraTrees translates to approximately 300 fewer misforecasted calls per day, equating to six agents better allocated daily. Over a year, this efficiency could save Coinbase upwards of \$300,000 in operational costs while enhancing customer satisfaction through reduced wait times. Yet, these gains are contingent on diligent maintenance. The crypto landscape is notoriously volatile, with sudden market shifts—such as a Bitcoin crash or a regulatory announcement—capable of altering call patterns overnight. This necessitates a proactive approach to model management, including regular retraining cycles and real-time monitoring to detect performance drift. Establishing a dedicated team or automated system to oversee these efforts will be crucial, ensuring that the model remains a reliable tool rather than a static solution.

Looking ahead, this guide serves as a springboard for further innovation. You might explore integrating real-time data feeds, such as social media sentiment from X or news aggregators, to enhance regime detection during unforeseen events like the FTX collapse. Additionally, experimenting with ensemble techniques—combining Prophet's seasonality expertise with ExtraTrees' non-linear prowess—could yield a hybrid model with even greater accuracy, potentially pushing the MAE below 600. The appendices provide a foundation for understanding model behaviors, but expanding them with detailed hyperparameter tuning logs or performance metrics across different market conditions could offer deeper insights for future iterations. Consider also simulating extreme scenarios, such as a 20% market drop, to stress-test the model's resilience and prepare for Black Swan events that defy historical patterns.

Implementation will require more than just deploying the model. Setting up an A/B testing framework involves running ExtraTrees alongside your existing system for 30-60 days, comparing metrics like MAE, customer wait times, and agent utilization. This dual-operation phase will reveal practical strengths and weaknesses, informing a phased rollout. Post-deployment, establish a retraining

cadence—monthly at minimum—using the latest data to adapt to new regimes. Invest in a monitoring dashboard that tracks daily MAE deviations, flagging anomalies when errors exceed 20% of the baseline, and integrate alerts to trigger human review. This infrastructure demands engineering resources, potentially a small ML ops team, to handle pipeline orchestration, drift detection, and quarterly performance reviews, ensuring the model evolves with Coinbase’s needs.

The success of this endeavor hinges on collaboration between data scientists, operations teams, and executives. Regular feedback loops—perhaps bi-weekly syncs—can align model outputs with business goals, such as optimizing agent schedules or reducing overtime costs by 20-30%. Documenting lessons learned, such as the failure of linear models to capture non-linearities or the runtime challenges of `TimeSeriesPipeline.ipynb`, will refine future projects. This guide is not just a technical manual but a strategic asset, empowering Coinbase to turn volatile market dynamics into predictable operational advantages. As you embark on this next phase, treat each iteration as an opportunity to enhance accuracy, efficiency, and adaptability, ensuring the journey’s end is merely the beginning of a more data-driven future.

Appendix: Model Performance Summary

Model	MAE	% Improvement vs. Baseline	Notes
Day-of-Week Baseline	920.58	0%	Our primary benchmark
Prophet	657.79	28.5%	Best Time Series Model
SARIMAX	682.38	25.9%	Strong Regime-Aware TS Model
ExtraTrees_100	608.72	33.9%	Overall Champion Model
LightGBM_100	671.47	27.1%	Strong ML Performer
Linear Regression	1350.29	-46.7%	Fails to capture non-linearity
Holt-Winters	1971.40	-114.1%	Fails in volatile conditions

Appendix 2: Business & Technical Model Definitions

Baseline Models

- **Mean Model**
 - **Business Analogy:** The simplest possible forecast. It's like a manager saying, "On an average day, we get about 4,500 calls, so let's plan for that every single day." It completely ignores all trends, seasons, or recent events.
 - **Technical Definition:** A baseline model that calculates the arithmetic mean of the entire training dataset. The forecast for every future time point is this single, constant mean value.
- **Median Model**
 - **Business Analogy:** Similar to the mean model but more robust to extreme days. It's like saying, "Some days are crazy and some are dead, but a typical, middle-of-the-road day has about 4,200 calls. Let's plan for that." This approach is less skewed by rare, massive spikes in volume.
 - **Technical Definition:** A baseline model that calculates the median (50th percentile) of the entire training dataset. The forecast for every future time point is this single, constant median value, making it less sensitive to outliers than the mean.
- **Day-of-Week Average Model**
 - **Business Analogy:** Assumes the business has a strong weekly rhythm. It's like a store manager staffing for next Friday based on the average sales of all previous Fridays. Simple and effective for cyclical operations.
 - **Technical Definition:** A baseline model that calculates the mean of the target variable for each day of the week from the training data. Predictions are made by mapping the corresponding day-of-week average to the forecast horizon.

Time Series Models

- **ARIMA (AutoRegressive Integrated Moving Average)**
 - **Business Analogy:** A forecaster who believes future performance is a combination of its own recent momentum (AutoRegressive part) and the lingering impact of its own past forecast errors (Moving Average part). It works best when the data is stable and doesn't have sudden shocks.
 - **Technical Definition:** A class of statistical models for analyzing and forecasting time series data that is made stationary through differencing (the 'Integrated' part). It uses autoregression to model the relationship between an observation and a number of lagged observations, and a moving average component to model the effect of past forecast errors.
- **SARIMAX (Seasonal ARIMA with eXogenous variables)**
 - **Business Analogy:** A forecaster that believes future call volume is a mix of its own recent trends, recent forecast errors, weekly patterns, and the influence of external factors like the stock market.
 - **Technical Definition:** A class of models that extends ARIMA by including a seasonal component and external variables (exogenous regressors) to explain the time series.
- **Holt-Winters (Exponential Smoothing)**

- **Business Analogy:** A forecaster who pays most attention to recent events but also accounts for an underlying trend (are things generally going up or down?) and a seasonal pattern (do calls always spike on Tuesdays?). Older data is given exponentially less weight.
- **Technical Definition:** A method that captures level, trend, and seasonality by applying exponential smoothing to each component. It uses smoothing parameters (alpha, beta, gamma) to determine how much weight is given to recent observations for each component.
- **Prophet**
 - **Business Analogy:** Like a sophisticated weather forecaster that understands seasons (weekly, yearly) but also incorporates special reports (market events, holidays).
 - **Technical Definition:** An additive regression model developed by Facebook that models time series as a combination of a non-linear trend, multiple seasonalities (using Fourier series), holiday effects, and the impact of external regressors.

Machine Learning Models

- **Linear Regression**
 - **Business Analogy:** A straightforward forecaster who assumes a simple, additive relationship between inputs and the outcome. For example, "Every \$1000 drop in Bitcoin adds 50 calls, and every 1-point jump in VIX adds 20 calls, period." It can't understand complex interactions.
 - **Technical Definition:** A statistical model that assumes a linear relationship between a dependent variable (the target) and one or more independent variables (features). It finds the best-fitting straight line (or plane) through the data points.
- **Random Forest**
 - **Business Analogy:** It's like asking hundreds of experts for their opinion, where each expert builds a detailed decision flowchart based on a random subset of information. The final forecast is the democratic average of all the experts' flowcharts, which is more robust than any single expert's opinion.
 - **Technical Definition:** An ensemble learning method that constructs a multitude of decision trees at training time. It outputs the mean prediction (regression) of the individual trees. It uses bootstrapping (bagging) to create data subsets for each tree and considers only a random subset of features for each split.
- **ExtraTrees (Extremely Randomized Trees)**
 - **Business Analogy:** Like polling a large group of diverse experts for their estimate, but each expert only looks at a random subset of information and makes a quick decision based on random thresholds. The average of all their "good enough" guesses turns out to be extremely accurate and less prone to overthinking.
 - **Technical Definition:** An ensemble learning method that fits a number of randomized decision trees on various sub-samples of the dataset. It is similar to Random Forest but differs in how it chooses splits: thresholds are drawn at random for each feature and the best of these randomly-generated thresholds is chosen as the splitting rule.
- **Gradient Boosting (including XGBoost & LightGBM)**
 - **Business Analogy:** A team of "specialist" forecasters who work sequentially. The first makes a simple prediction. The second focuses only on correcting the first one's

mistakes. The third corrects the second's mistakes, and so on. Each new specialist makes the team slightly smarter by focusing on the hardest-to-predict cases.

- **Technical Definition:** An ensemble machine learning technique that builds models in a stage-wise fashion. It fits a sequence of weak learners (typically decision trees) to the residuals (errors) of the prior models. It optimizes a differentiable loss function using a gradient descent-like procedure. XGBoost and LightGBM are highly efficient and regularized implementations of this concept.